Лабораторна робота B15: std::transform

Виконав: Остапенко Роман К-24 Компілятор: g++ 16.2.0, збирання RELEASE (-O0 -DNDEBUG / -O3 -DNDEBUG)

1. Постановка

Досліджено швидкодію std::transform (вхідний vector<double> -> вихідний вектор) для двох перетворень: швидкого x·2.5+pi (кілька тактів на елемент, обмежене пропускною здатністю пам'яті) та повільного — 200 ітерацій

r = sin(r)·cos(r)+sqrt(|r|) (сотні наносекунд на елемент, обмежене процесором). Порівнювались: 1) алгоритм без політики; 2) політики seq, par, par_unseq; 3) власний алгоритм customTransform: вибірка ділиться на K майже рівних частин (остача - в останню), кожна обробляється послідовним std::transform в окремому std::jthread; результат кожен потік пише у свій діапазон вихідного вектора, тому окреме зведення не потрібне, а синхронізація зводиться до join() у деструкторах jthread. K = 1…24 (* 2 - кількість апаратних потоків).

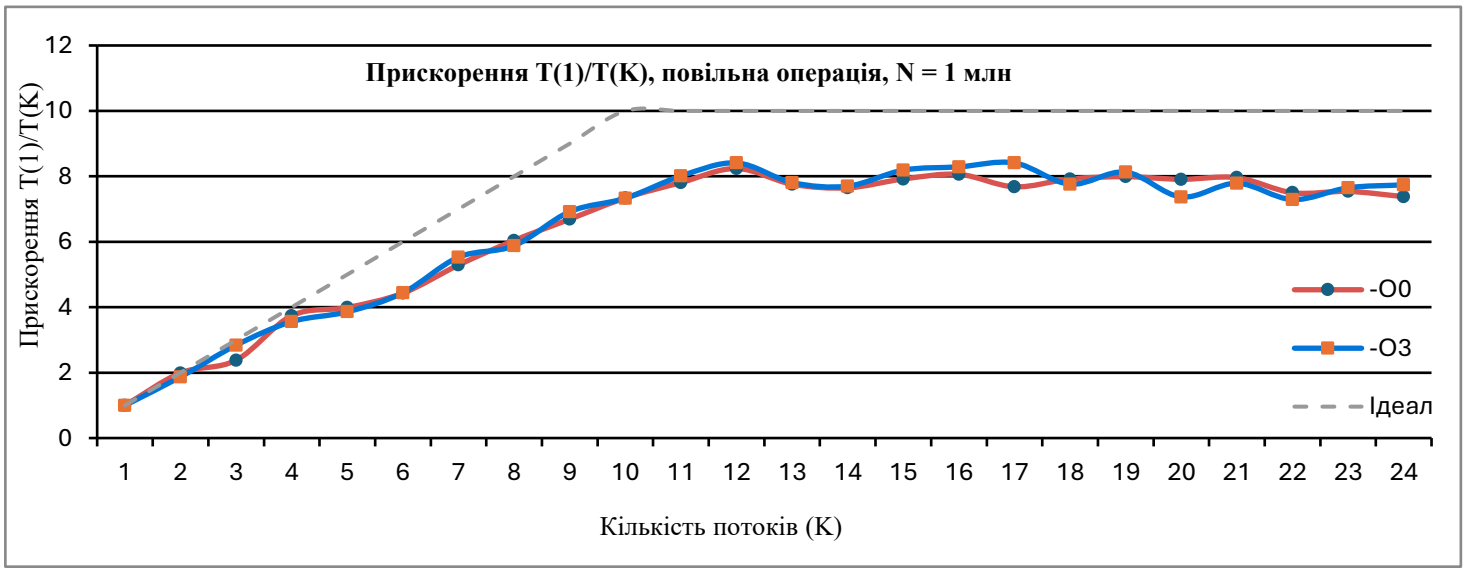
Дано: 3 послідовності (10 тис, 100 тис, 1 млн) з <random> (mt19937, uniform_real_distribution(1, 100)), згенеровані заздалегідь і збережені у data\. Однаковий набір файлів використано для всіх політик, усіх K та обох рівнів оптимізації. Час-std::chrono::high_resolution_clock, мс, один прогін. Процесор: 12 логічних потоків (std::thread::hardware_concurrency()). Сирий вивід: results/release_O0.txt, results/release_O3.txt.

2. Бібліотечний алгоритм: політики та рівень оптимізації (мс)

Операція	N	RELEASE -O0					RELEASE -O3				
		без пол.	seq	par	par_unseq	власний	без пол.	seq	par	par_unseq	власний
швидка	10 тис	0.032	0.048	0.047	0.050	0.032 (K=1)	0.010	0.012	0.012	0.016	0.013 (K=1)
швидка	100 тис	0.325	0.500	0.460	0.471	0.311 (K=1)	0.158	0.134	0.124	0.159	0.124 (K=1)
швидка	1 млн	3.27	4.72	4.74	4.88	0.858 (K=17)	1.20	1.44	1.38	1.74	0.640 (K=14)
повільна	10 тис	49.7	48.8	48.6	48.8	6.21 (K=12)	47.4	47.3	46.7	47.8	6.07 (K=24)
повільна	100 тис	490	490	488	489	60.9 (K=24)	466	465	474	475	57.5 (K=23)
повільна	1 млн	4897	4893	4890	4891	593 (K=12)	4673	4664	4655	4643	553 (K=17)

3. Власний паралельний алгоритм: залежність часу від K (мс, N = 1 млн; найкраще значення виділено)

K	1	2	3	4	6	8	10	11	12	13	14	16	17	18	20	24
повільна, -O0	4888	2451	2055	1306	1104	809	666	626	**593**	630	639	606	637	617	618	662
повільна, -O3	4653	2486	1641	1309	1048	791	635	581	553	596	605	561	**553**	599	632	601
швидка, -O0	3.25	1.87	1.45	1.24	1.05	1.18	0.929	1.02	0.958	1.01	1.10	0.889	**0.858**	0.874	0.891	0.965
швидка, -O3	1.44	1.11	1.00	1.20	1.07	0.849	0.827	0.668	0.671	0.640	**0.640**	0.683	0.711	0.694	0.795	0.863
швидка, -O3, 10 тис.	**0.013**	64.3	0.401	0.457	0.622	0.394	0.357	0.381	0.407	0.441	0.473	0.507	0.558	0.578	0.662	0.759



Знайдене найкраще K (вивід програми, «Best K»):

• повільна операція: K = 12 (-O0) та K = 17 (-O3) для 1 млн; для 100 тис. - 24 / 23; для 10 тис. - 12 / 24. Прискорення при K = 12: *8.2 (-O0), *8.4 (-O3).

• швидка операція: K = 17 / 14 для 1 млн, але K = 1 для 10 тис. і 100 тис.

Значення K > 12 іноді формально перемагають, але для 100 тис. і 1 млн відрив від K = 12 становить ≤ 1–2 % (напр.: -O3, 1 млн: K = 12 -> 553 мс, K = 17 -> 553 мс). Для 10 тис. весь прогін триває приблизно 6–8 мс, і розкид між сусідніми K (до десятків відсотків) - це шум одного вимірювання, а не закономірність.

4. Висновки

1) Найкраще K та кількість потоків процесора. Для обмеженої процесором (повільної) операції час падає майже як T(1)/K до K = 12, тобто до кількості логічних потоків процесора: при K = 12 прискорення *8.4–*8.2 замість ідеальних *12. Отже, оптимальне K ≈ кількості апаратних потоків: 12; далі виграшу немає. Для швидкої операції оптимум залежить від обсягу: на 1 млн елементів найкраще K ≈ 14–17 (≈ 12), але прискорення лише *2.3–*3.8, бо операція впирається в пропускну здатність пам'яті, а не в ядра; на 10–100 тис. елементів найкраще K = 1: уся робота триває 0.013–0.124 мс, а створення навіть одного додаткового потоку коштує більше.

2) Час можемо описати моделлю T(K) ≈ T1/min(K, P) + c·K, де P = 12 (в нашому випадку), c — вартість створення й приєднання одного потоку. Коли K ≤ P, домінує перший доданок (гіпербола). Коли K > P, перший доданок перестає зменшуватись, і час росте **лінійно** по K. Найчистіше це видно на швидкій операції з 10 тис. елементів: від K = 3 до K = 24 час зростає майже рівномірно. Для повільної операції лінійний доданок малий порівняно з T1/P, тому після K = 12 спостерігається плато (тобто: середнє для K = 13…24 при -O3: 594 мс проти 553 мс при K = 12). Стрибок при K = 12+1 спричинений нерівномірним навантаженням: 12+1 рівних частин на 12 потоках означає, що одна частина виконується у другу хвилю.

3) Політики виконання. У GCC політики par/par_unseq виконуються паралельно лише з бекендом Intel TBB. Без TBB вони виконуються послідовно, і це підтверджують вимірювання: seq, par і par_unseq дають однаковий час (напр. -O3, повільна, 1 млн: 4664 / 4655 / 4643 мс). При -O0 seq навіть повільніший за версію без політики на швидкій операції (4.72 проти 3.27 мс), бо додаткові шари обгорток PSTL не вбудовуються (не інлайняться). При -O3 ця різниця майже зникає. Тому в цій конфігурації реальне прискорення дає лише власний алгоритм на std::jthread.

4) Рівень оптимізації. Для швидкої операції -O3 прискорює бібліотечний transform у ×2.7 (1 млн: 3.27 -> 1.20 мс), бо цикл, ітератори та виклики стають компактним машинним кодом. Для повільної операції виграш лише ×1.05: майже весь час витрачається всередині вже скомпільованих оптимізованими функцій sin/cos/sqrt, на які прапорці нашого коду не впливають. Положення оптимального K від рівня оптимізації не залежить.

5) Похибки. Кожне значення отримано за один прогін, тому окремі точки мають викиди (напр. -O3, 10 тис., K = 2: 64.3 мс — перше створення потоків у процесі). Тенденції від цього не змінюються, бо однакові на всіх трьох обсягах і обох рівнях оптимізації.